Université Mohammed V de Rabat

Ecole Supérieure de Technologie de Salé

Back-End Et APIs

Pr R.ALAOUI

Programme

Architectures Réparties (Java EE)

- Séparation logique entre couche présentation, couche métier et couche de persistance.
- Bonnes pratiques pour une architecture modulaire et évolutive.
- Utilisation de serveurs d’application Java EE (ex. Tomcat, GlassFish).

Programmation Client-Serveur avec Servlets et JSP

- Développement d'applications web dynamiques.
- Gestion des requêtes HTTP, sessions, cookies et filtres.
- Interaction avec les pages JSP pour l’affichage côté client.

Persistance avec JPA (Java Persistence API)

- Mapping objet-relationnel (ORM) avec **Hibernate** et **EclipseLink**.
- Configuration de l’entity manager et annotations JPA.
- Requêtes JPQL, gestion du cycle de vie des entités.
- Transactions, relations entre entités et performances.

Introduction aux Web Services : SOAP vs REST

- Principes fondamentaux de SOAP (XML, WSDL, enveloppes).
- Introduction à REST : HTTP, URI, JSON/XML, statelessness.
- Comparaison : usages, formats, complexité et évolutivité.

Caractéristiques et Architectures des Services Web

- Types de services web : RPC, Document-based.
- Protocole HTTP, sécurisation, authentification.
- Modèles d'interopérabilité entre services distribués.

Fondamentaux des APIs RESTful : principes et bonnes pratiques

- Utilisation des méthodes HTTP (GET, POST, PUT, DELETE).
- Gestion des erreurs, statelessness, pagination, versioning.
- Structuration des URI et convention REST.

Environnements et outils pour les Web Services

- Outils de test (Postman, SoapUI).
- Frameworks de développement (Spring Boot, JAX-RS).
- Documentation des APIs (Swagger/OpenAPI).

Introduction aux Microservices

- Définition, objectifs et cas d'usage.
- Principe de découplage et indépendance des services.
- Exemple de conception d'un microservice en Java..

Développement avec Spring Boot

- Introduction au framework Spring Boot.
- Création rapide d'applications web et REST.
- Intégration avec JPA (Spring Data JPA).
- Injection de dépendances, gestion des configurations.
- Tests unitaires et intégration avec Spring Boot Test.

Comparaison avec les architectures monolithiques

- Avantages/inconvénients de chaque approche.
- Évolutivité, maintenabilité, déploiement, résilience.

Conteneurisation avec Docker et Orchestration avec Kubernetes

Université Mohammed V de Rabat

Ecole Supérieure de Technologie de Salé

Chap 1

Architectures Réparties

Pr R.ALAOUI

Plan

Plateforme J2EE: Architectures Réparties Pour les Bases de Données

- la plateforme Java EE
- les architectures adaptées au développement de projets Web
- Bibliothèques et API Java EE

J2EE Objectifs

• Faciliter le développement de nouvelles applications à base de composants

• Intégration avec les systèmes d'information existants

• Support pour les applications « critiques » de l'entreprise **tolérance aux pannes**, **montée en charge, sécurité**...

C’est quoi J2EE ?

Une **technologie**:
– Le langage Java
– La machine virtuelle (JVM)
– Des APIs (le JDK + APIs applicatives)
– Des serveurs respectant le standard Java EE (JSR)

JEE inclus dans les Systèmes distribués (Architectures Réparties)
- Division des traitements en modules indépendants
- plus disponible
- plus évolutif
- plus maintenable

Fourmis

Architectures Réparties : petits, rapides et facilement adaptables

Dinosaure

Systèmes monolithique : gros, lents et inadaptables

C’est quoi J2EE ?

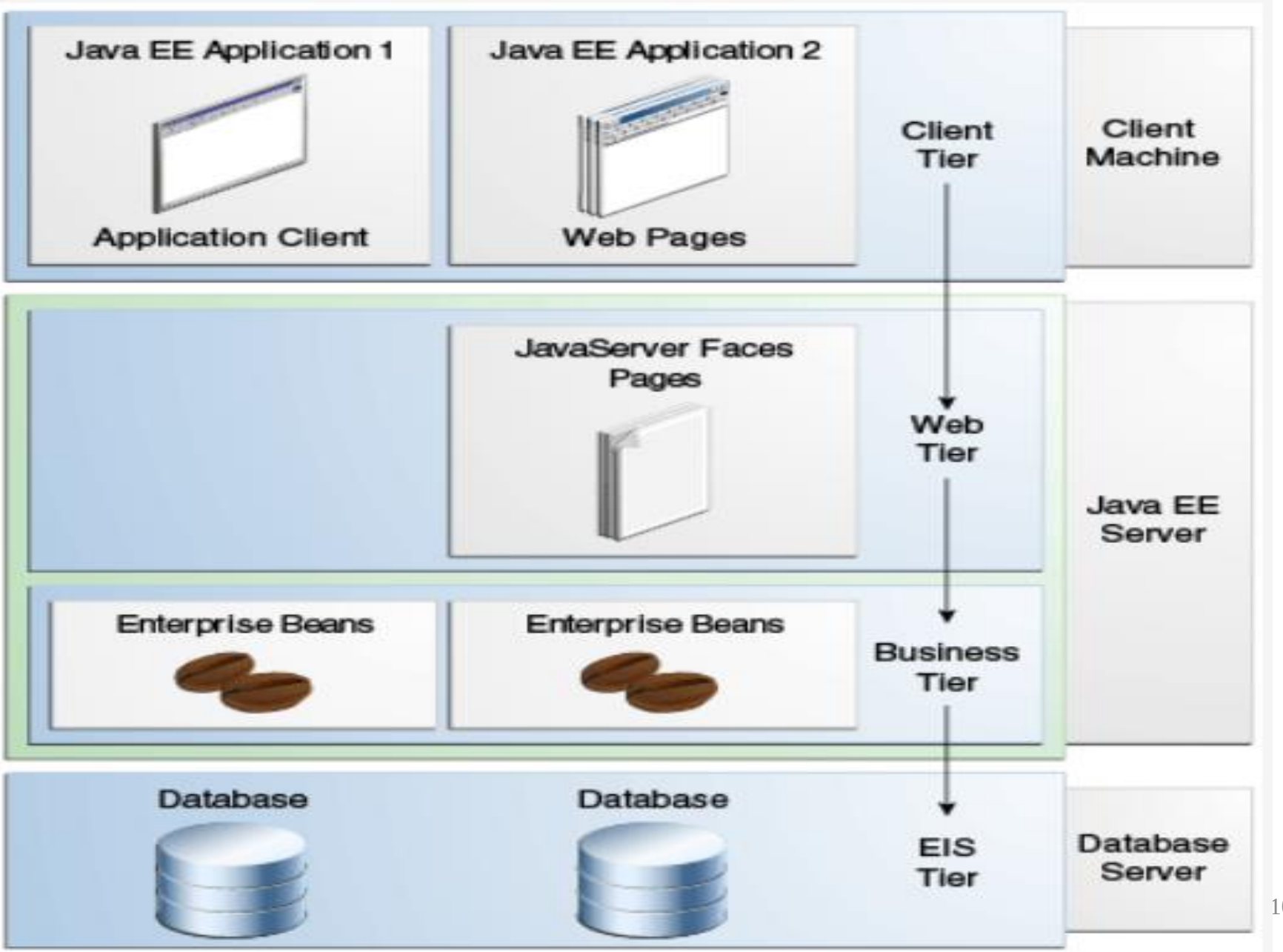
Un modèle de développement:

– **Développement en tiers (multi-tiers)** : applications découpées logiquement (correspondance avec le déploiement : clients, serveurs, SGBDs,...)

– **Ce modèle partitionne le travail en 2 parties** :

- Les aspects métiers/présentation, à la charge du développeur
- Les services standards fournies par la plate-forme Java EE

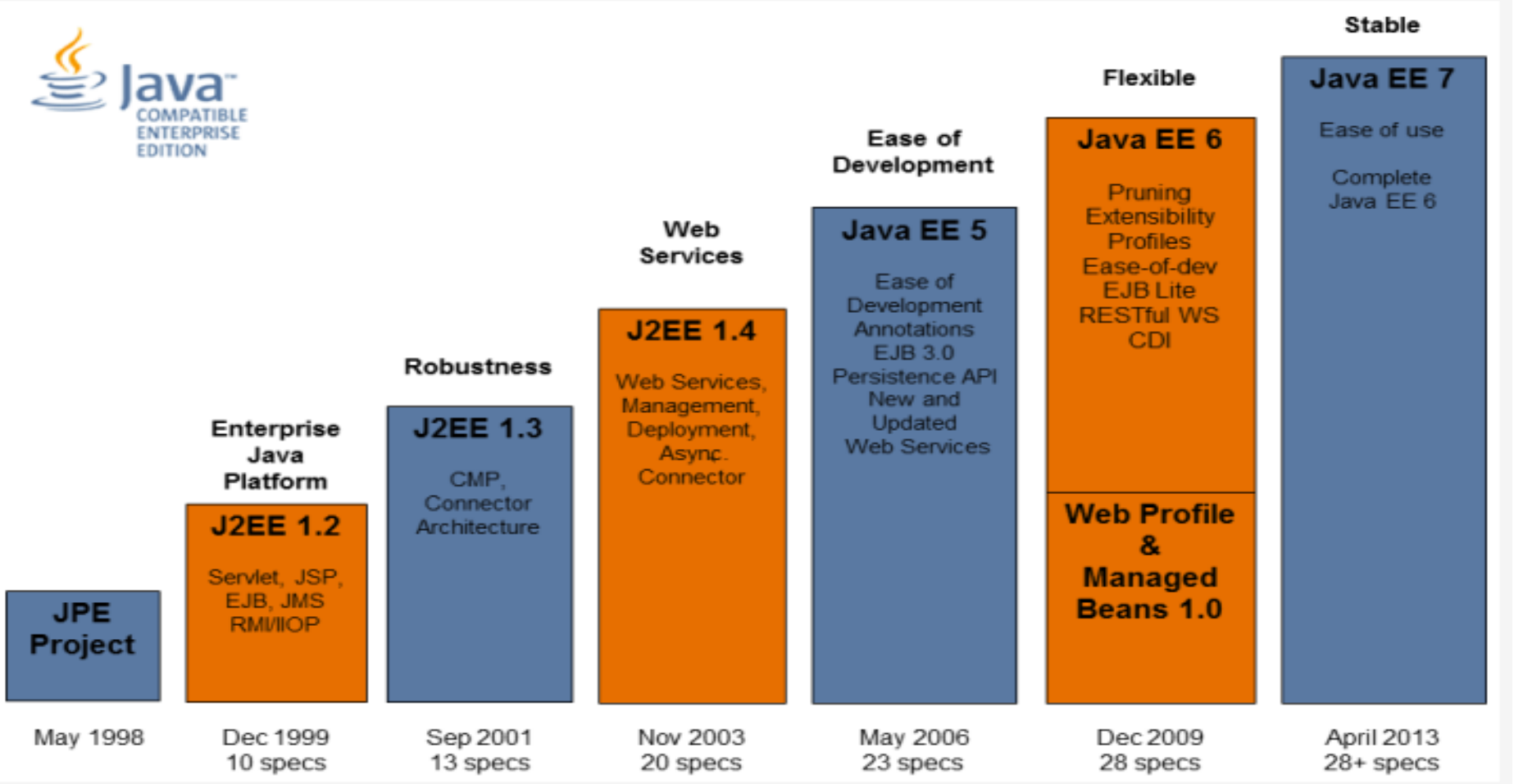
C’est quoi J2EE ? Applications multi-tiers



Présentation et historique

Sun Microsystems, désormais **Oracle Corporation**, propose pour cela la plateforme **Java Enterprise Editio**n ou **Java EE**, véritable **référence** pour le **développement d'applications supportant** en **standard**
la programmation,
la persistance des données,
les transactions,
la sécurité
le déploiement de projets sur serveur applicatif.

Présentation et historique



En 2018, Le nouveau nom, Jakarta EE, est le deuxième changement majeure de nom de JEE. Jakarta EE a été optimisé pour le cloud computing. Selon la fondation Eclipse, l'un des principaux objectifs de Jakarta EE est d'accélérer le développement d'applications d'entreprise pour le Cloud Computing.

Présentation et historique J2EE

La plateforme Java EE propose un **développement simplifié**, **rapide** et **robuste** reposant sur la **configuration par annotations**, directement dans les fichiers source.

- La première version standard J2EE 1.2 a été développée par Sun en **1999** et contenait **10 spécifications** .

- La version J2EE 1.3 sortie en **2001** n'apportait pas de nouveautés essentielles mais plutôt des améliorations de la plateforme à la suite des demandes utilisateur, comme la **configuration par fichier XML**.

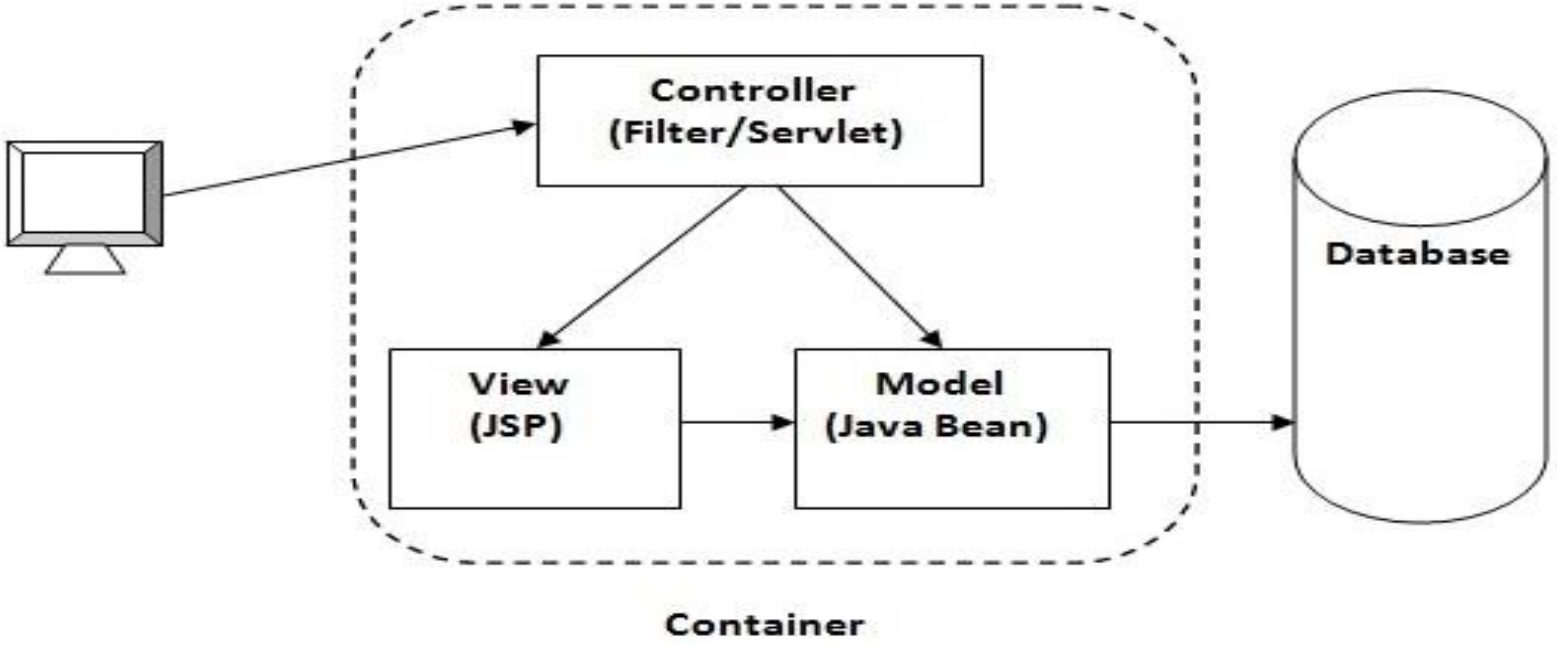
- La version J2EE 1.4 proposait **20 spécifications** lors de sa sortie fin **2003**. Cette version apportait le support des **services Web** et **des EJB version 2.1**.
(version conséquente était difficile à utiliser, tester et déployer.

Présentation et historique J2EE

Architecture MVC

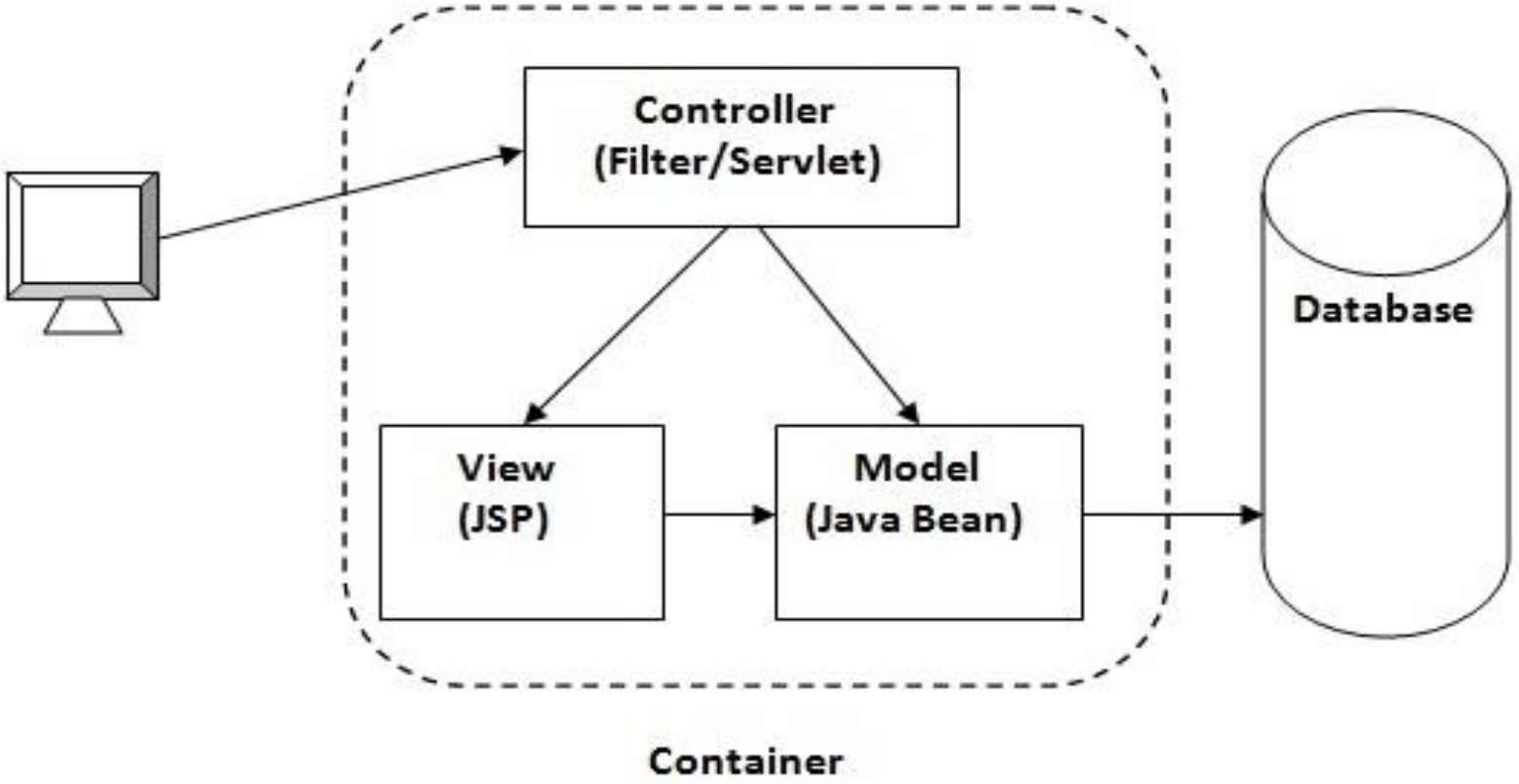
MVC est une manière idéale qui consiste à modéliser un composant selon trois parties indépendantes :

Le modèle (Model) qui stocke les données qui définit la composante.
(En général; Les classes du modèle sont des Java Bean correspondant aux champs des tables de la base de donnée)



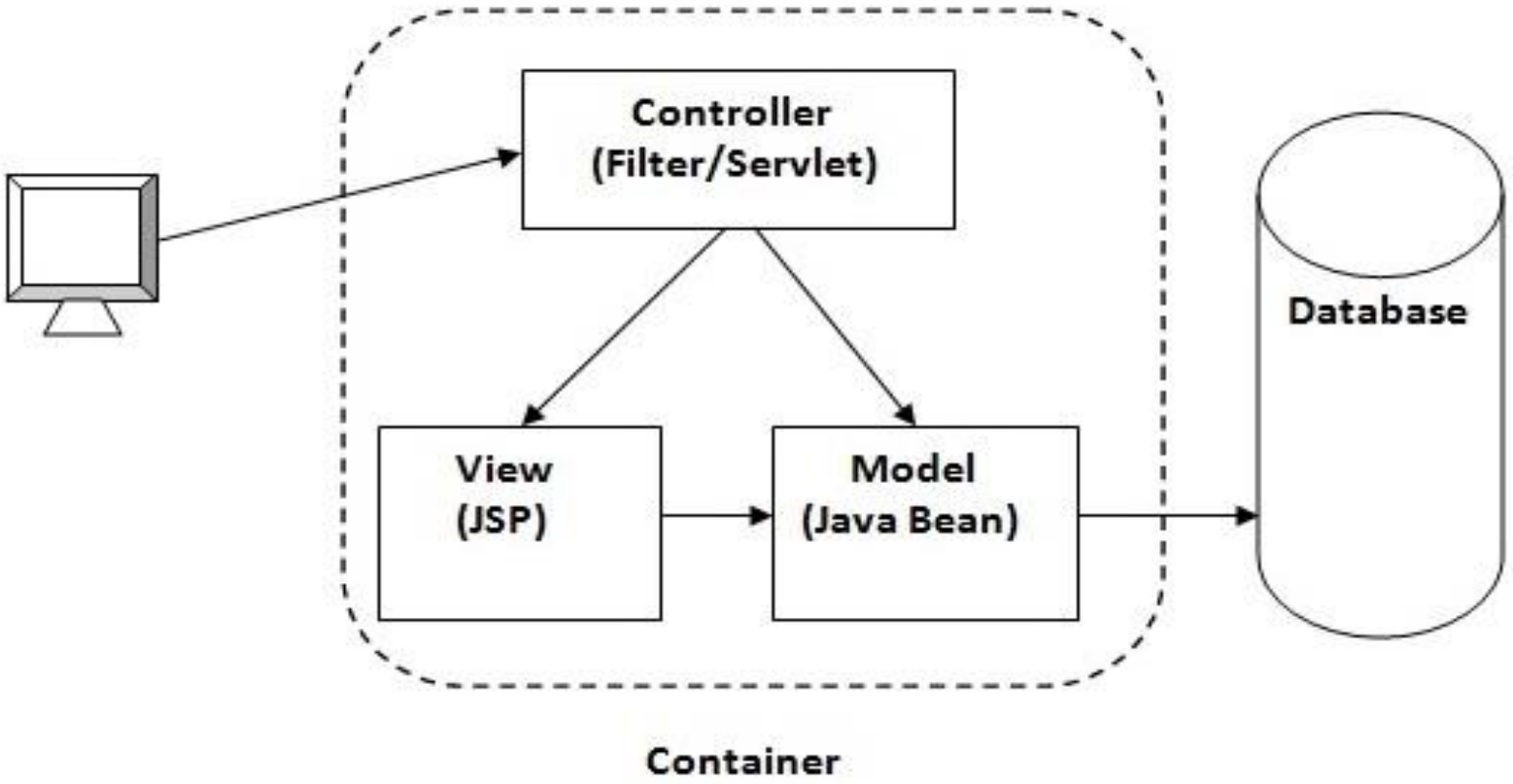
Présentation et historique J2EE

La vue (View) : La vue c'est ce que voit l'utilisateur
crée la représentation visuelle de la pièce à partir des données du modèle.
(En général; Les vues sont des pages jsp, xhtml…)



Présentation et historique J2EE

Le contrôleur (Controller) qui traite l'interaction de l'utilisateur avec le composant et modifie le modèle et/ou la vue, en réponse à une action de l'utilisateur selon les besoins.
Le **contrôleur** est une **Servlet** : **intermédiaire** entre la **vue** et le **modèle**.
Il **alimente** la **vue** par les **données fournit** par le **modèle**.



Présentation et historique J2EE

Problème: Les **développements par couches**, de type Modèle Vue Contrôleur (**MVC**), ne facilitent pas la compréhension de l'ensemble, et les technologies Servlet et JSP ne sont pas adaptées pour un développement rapide.

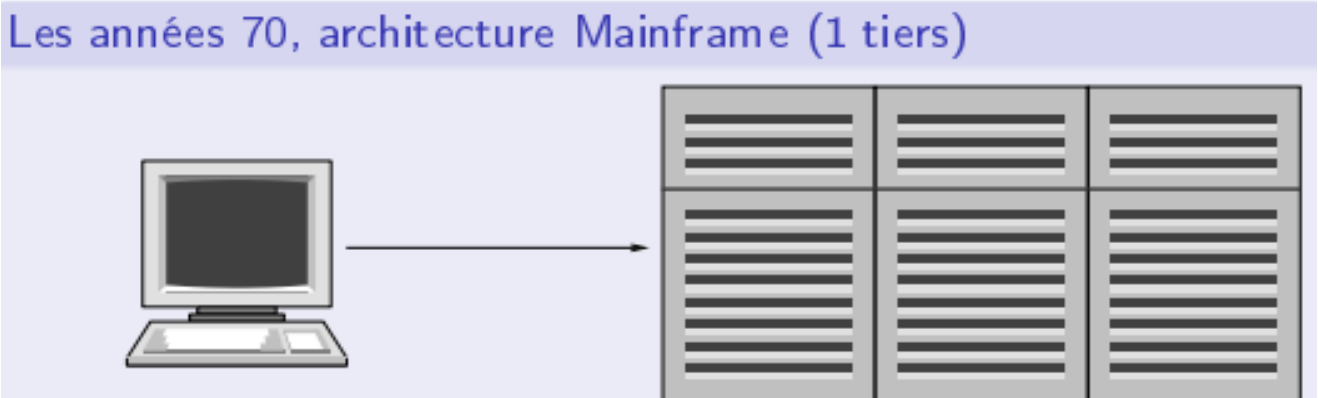
Solution: quelques **frameworks** ou **outils spécialisés** voyaient le jour, comme **Struts et Spring** pour la partie développement serveur, et **Hibernate ou TopLink** pour la persistance.

Composants et architectures
Application monolithique

Une **application monolithique** est un **programme constitué** d'un **seul bloc** et **s'exécutant** sur une **seule machine**.

Ces applications sont généralement **utilisées** dans le **domaine du temps réel** ou bien au **sein d'applications demandant** de grandes **performances**.

Elles restent également **omniprésentes dans le monde du grand public**, étant utilisées en **standalone sur les machines personnelles**.



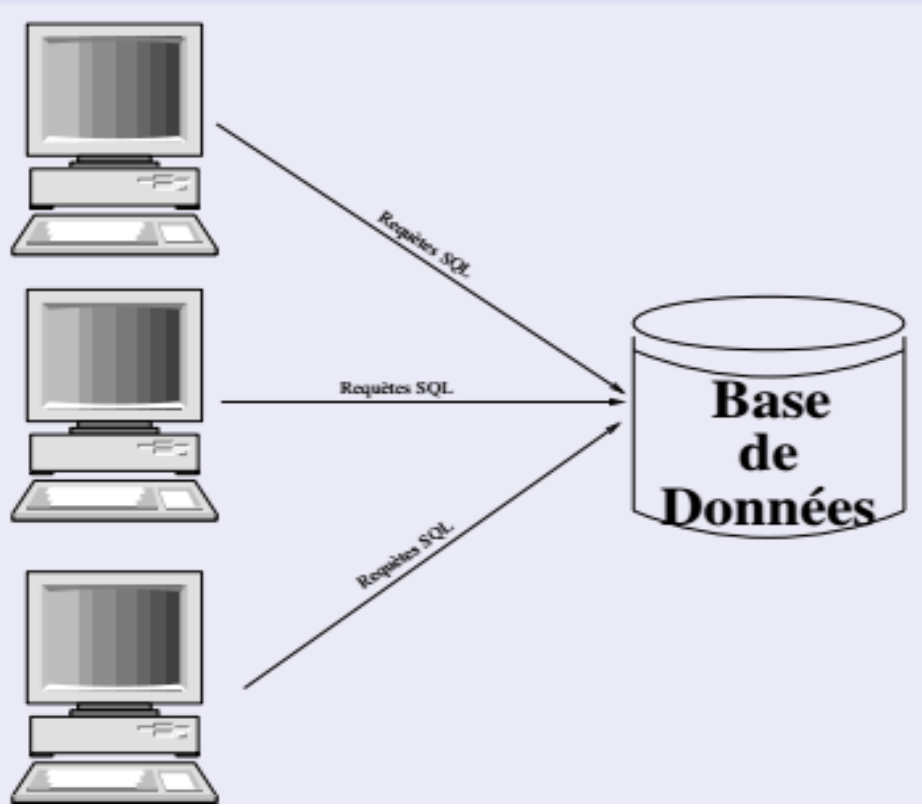
Composants et architectures
Architectures « client/serveur »

Dès l'apparition des réseaux, ces **applications** ont cherché à **évoluer** et ont abouti à des **architectures dites « client/serveur »,** permettant de **séparer la partie cliente** et de **regrouper la partie applicative sur un serveur**.

Les années 80, architecture 2 tiers

Principe

Architectures Client-Serveur nécessite la création d'un **protocole de communication** entre le **client et le serveur**.

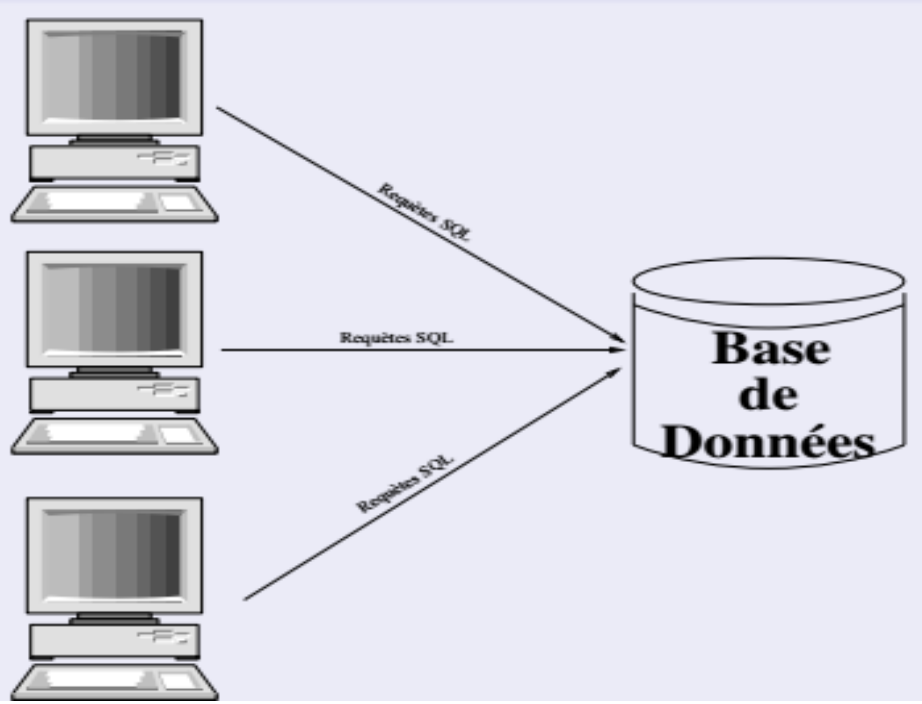


Composants et architectures
Problème Architectures « client/serveur »

- **Pour les systèmes d'information** d'entreprise ces **solutions restent trop limitées :**

Handicape majeur vue le **<u>manque de séparation entre les différents éléments</u>s qui le constituent**.

- Manque de standards qui **a poussé la communauté** au concept de « brique » et de **séparation des tiers** afin **d'optimiser leurs développements**.



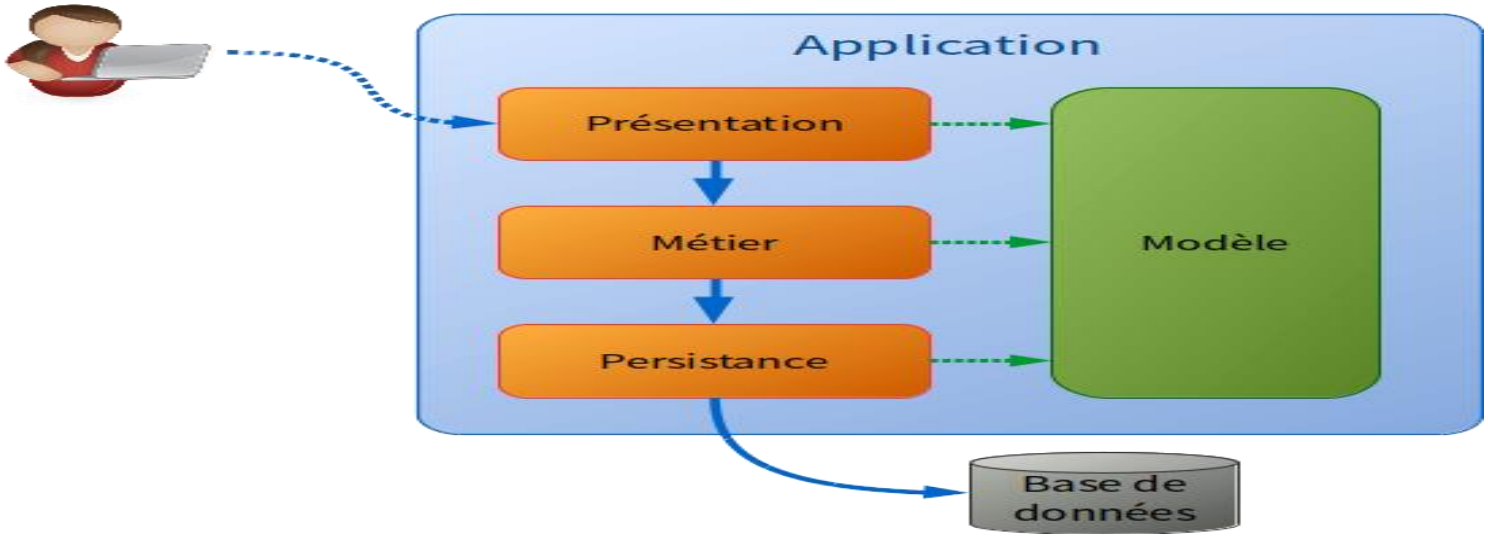
Composants et architectures
Application multi-tiers

Dans le milieu professionnel, les **applications doivent** être plus **robustes et travaillent** généralement sur de **gros volumes de données**.

Elles doivent, de plus, **connecter différents départements** au sein **même d'une entreprise**, ou avec d'autres entreprises.

La maintenance et la stabilité de **ces applications** sont donc des **priorités pour les architectes et les développeurs**.

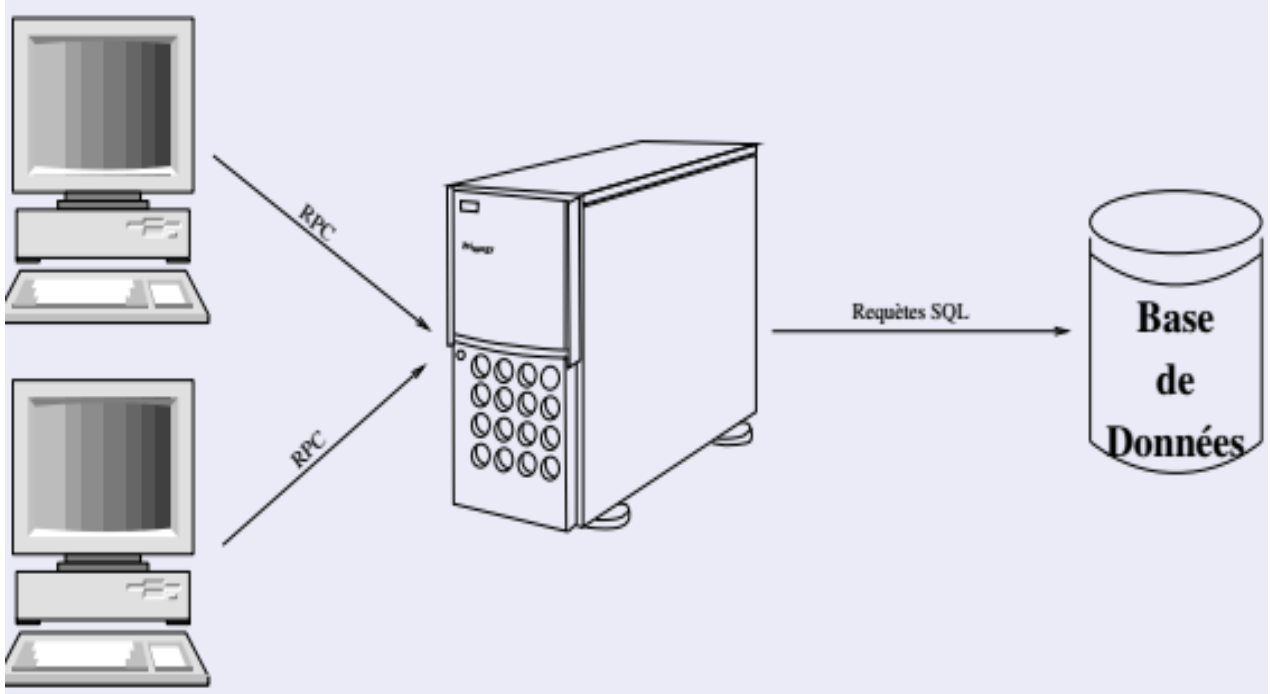
Différents modèles existent. Le **plus connu** est sans doute le **modèle 3-tiers**



Composants et architectures
Application multi-tiers: Modèle 3-tiers:

- **Plusieurs applications différentes** avec une **même logique métier**

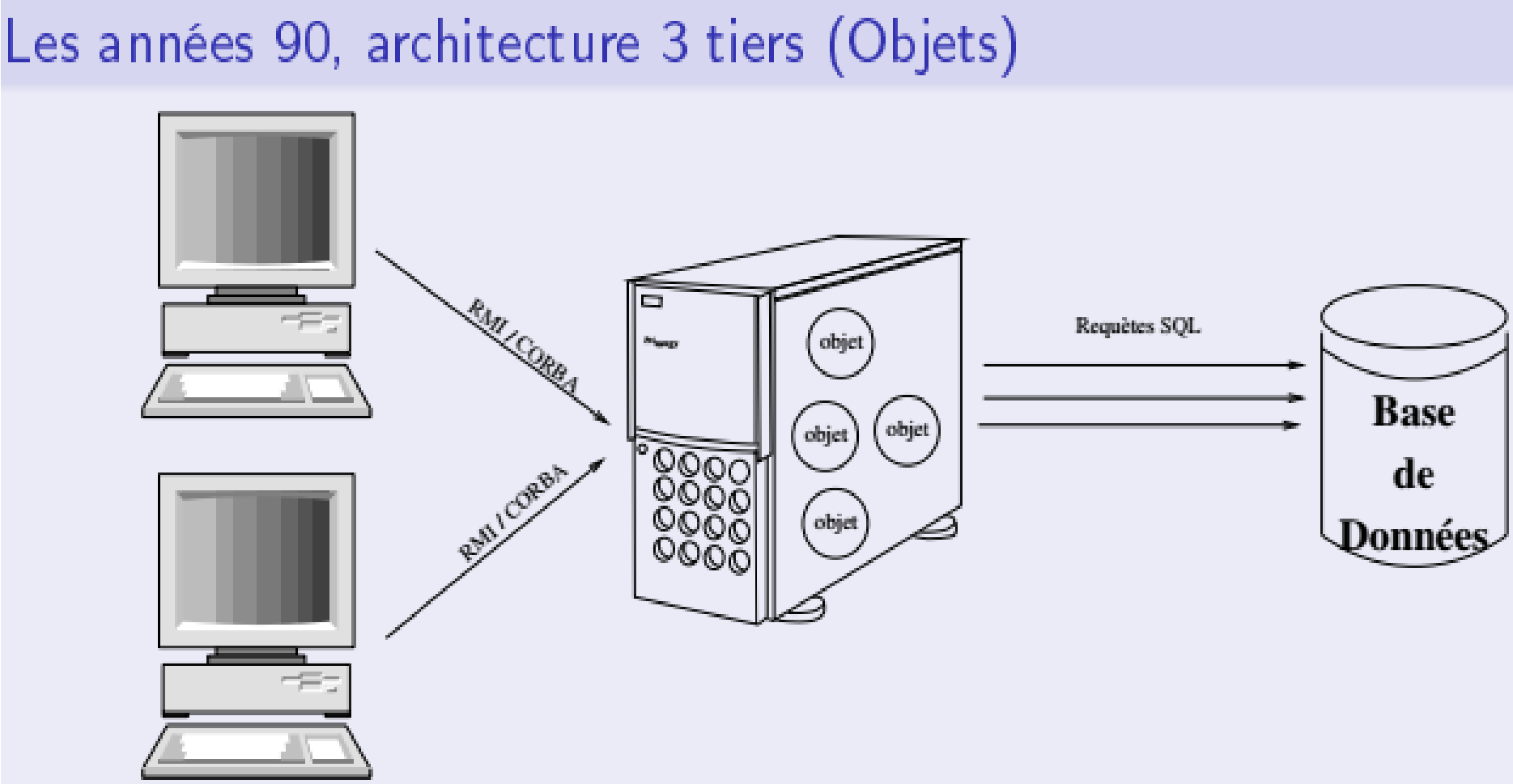
- **Mettre en place** facilement des **applications « distribuées »** dans un **environnement hétérogène**.



Fin des années 80, architecture 3 tiers (RPC)

Composants et architectures
Application multi-tiers: Modèle 3-tiers:

Architecture 3 tiers objet



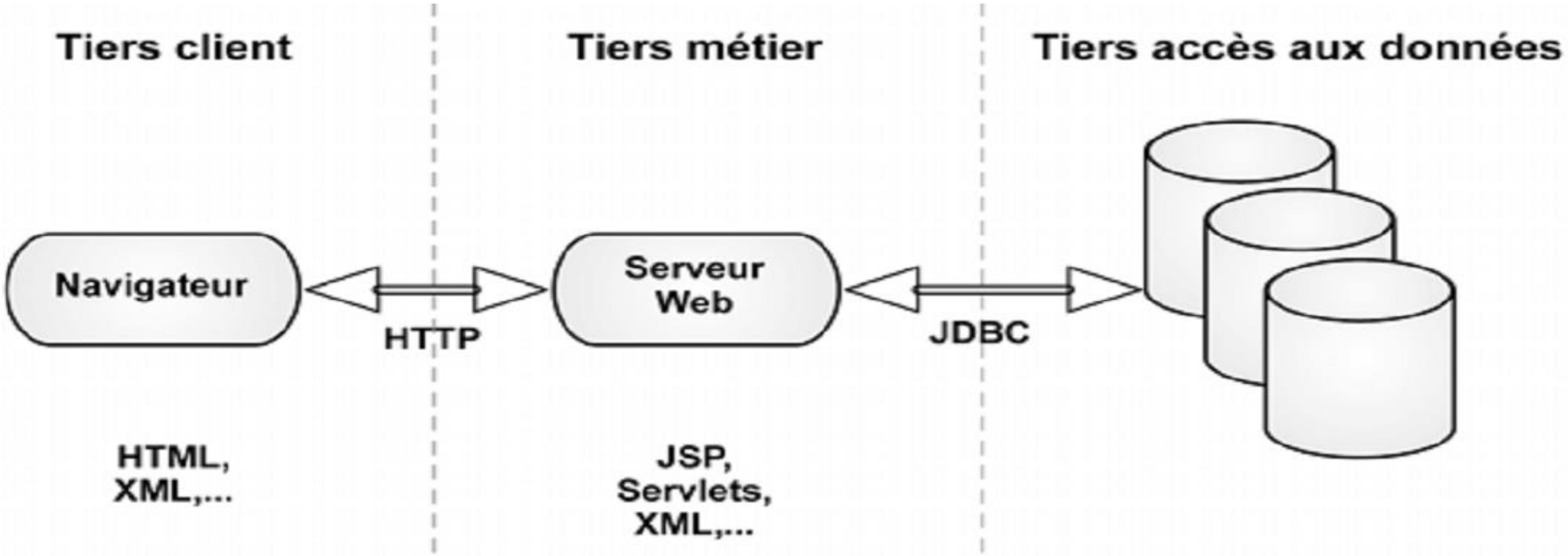
Composants et architectures

Application multi-tiers : Modèle 3-tiers

<u>Navigateur</u>: pour représenter l'application sur la machine cliente,
<u>Serveur web</u>: Pour la gestion de la logique de l'application
<u>Serveur de base de données</u>: Pour le **stockage des données**.

L'architecture 3-tiers est **divisée en trois niveaux** :

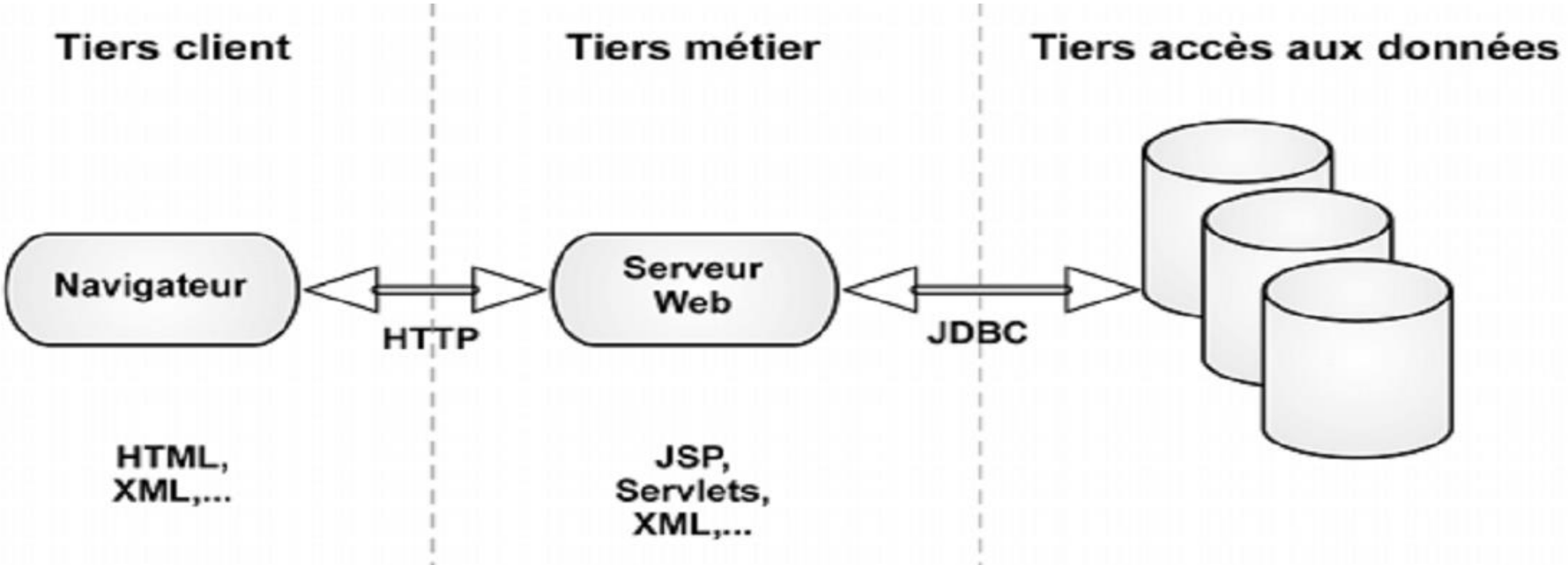
• **<u>Tiers client</u>** qui **correspond** à la **machine sur laquelle l'application cliente est exécutée.**



Composants et architectures
Application multi-tiers : Modèle 3-tiers

• **<u>Tiers métier</u>** qui **correspond** à la **machine sur laquelle l'application centrale est exécutée.**

• **<u>Tiers accès aux données</u>** qui **correspond** à la **machine gérant le stockage des données.**



Composants et architectures
Le modèle 3-tiers

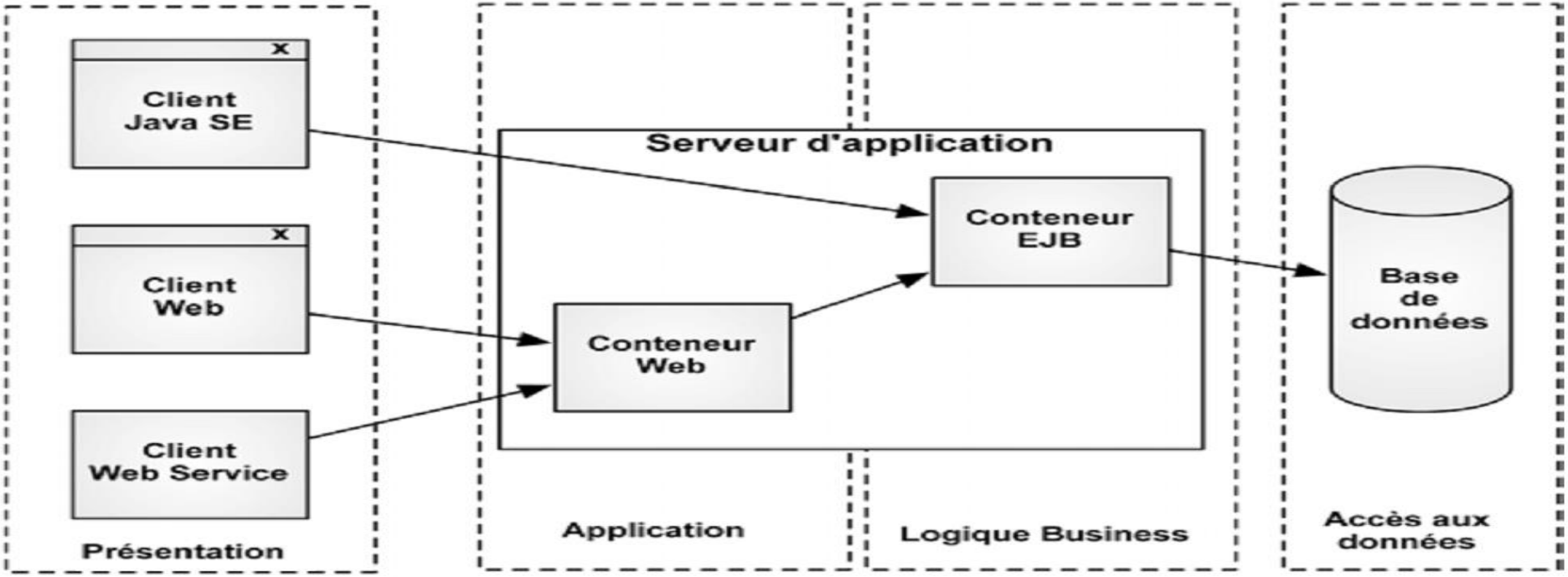
Problème : Non Séparation entre le client, l'application et le stockage.

Aucune séparation n'est faite au sein **même de l'application**, qui **gère aussi bien la logique métier que la logique fonctionnelle et l'accès aux données.**

Besoins: Les architectures qui **demandent de nombreuses ressources** (**plusieurs bases de données, clustering, interconnexion** avec **d'autres systèmes, différents clients**…) **cherche autre solution!!**

Composants et architectures
Le modèle n-tiers

L'architecture n-tiers est une **généralisation** du **modèle 3-tiers** qui **prend en compte l'évolutivité du système** et **évite les inconvénients de l'architecture 3-tiers** vus précédemment.

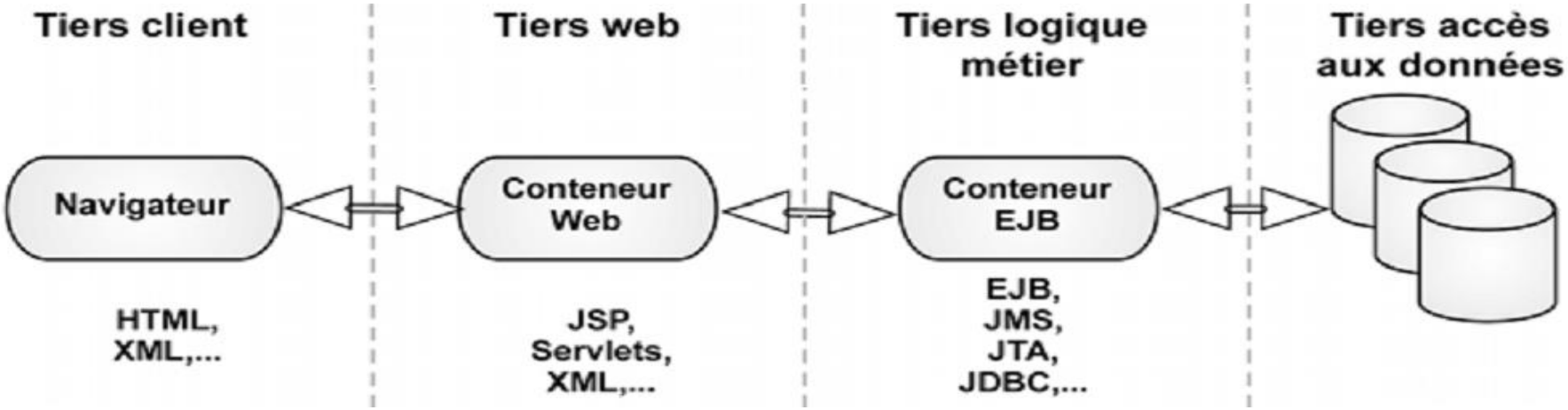


Dans la pratique, **on travaille** généralement avec un **tiers permettant de regrouper** la **logique métier de l'entreprise**. **Ce tiers** peut alors être **« appelé » par différentes applications clientes**.

Composants et architectures
Le modèle n-tiers

L'interconnexion entre ces tiers est le **grand problème de cette architecture**.

<u>Solution</u>: Integrer des frameworks, qui offrent un **ensemble de composants standards JEE** permettant d'**interconnecter les différentes technologies** sans difficulté.



Composants et architectures

Le modèle n-tiers

Application en couches

1. Couche présentation : gère le rendu des données

Frameworks MVC Struts , JSF, Spring

2. Couche Application: Classes simple Java et design patterns

3. Couche métier (Business Layer)

- Service
- Métier : contrôleur métier
- Technique
- Objet Métier persistant (EJBs)

4. Couche d'accès aux données (Persistance layer) permet **d'accéder aux données**

- Liaison avec entrepôt de données
- Gestion des opérations CRUD : Create, Read, Update, Delete.

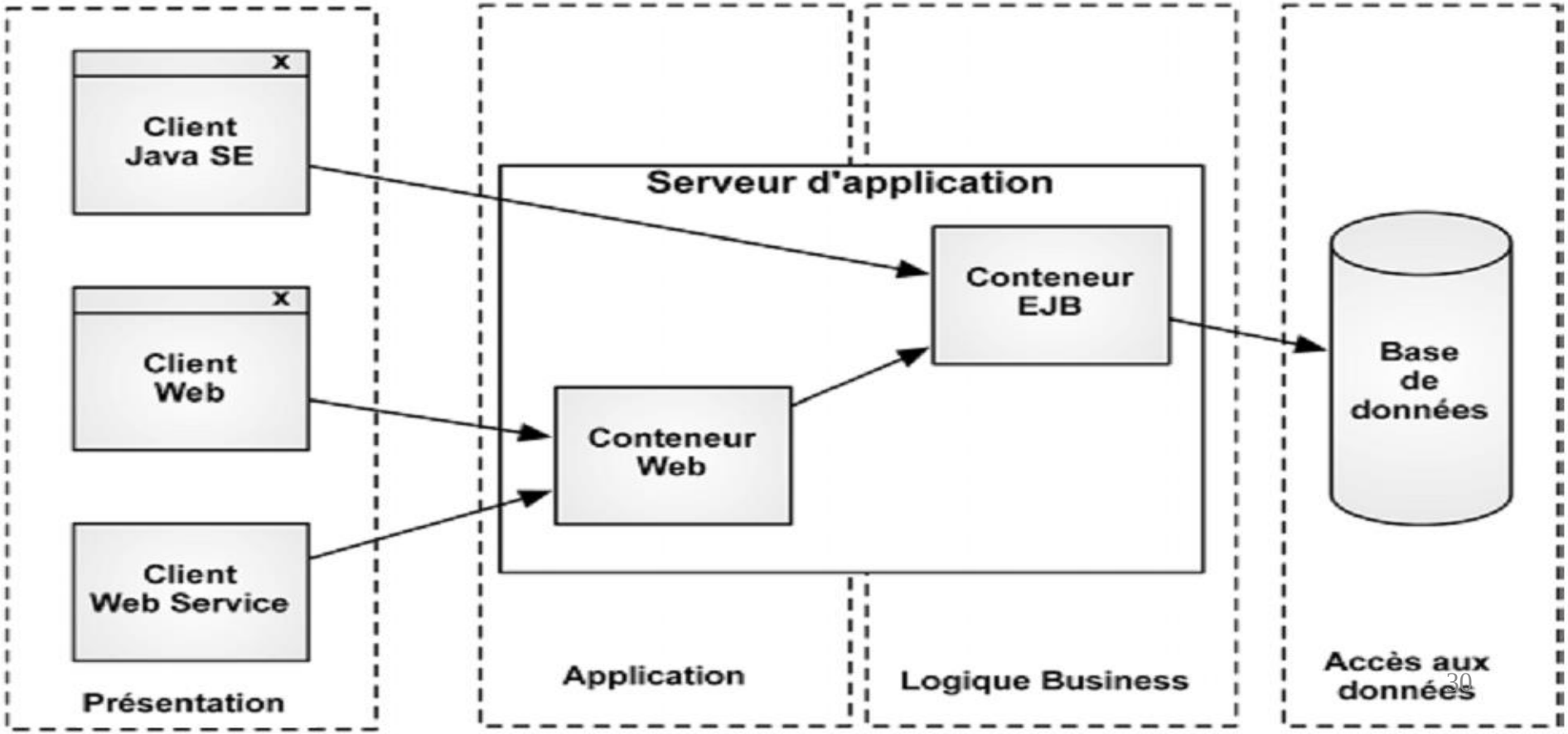
Données (Physical Layer)

- Entrepôt des données de l'application
- Schema modele physique de base de données, schema XML,…

DÉTAILS DES COUCHES

L'architecture logicielle d'une application est, constituée des couches suivantes (voir fig) :

- **présentation (client),**
- **application,**
- **métier, ou logique business,**
- **accès aux données.**

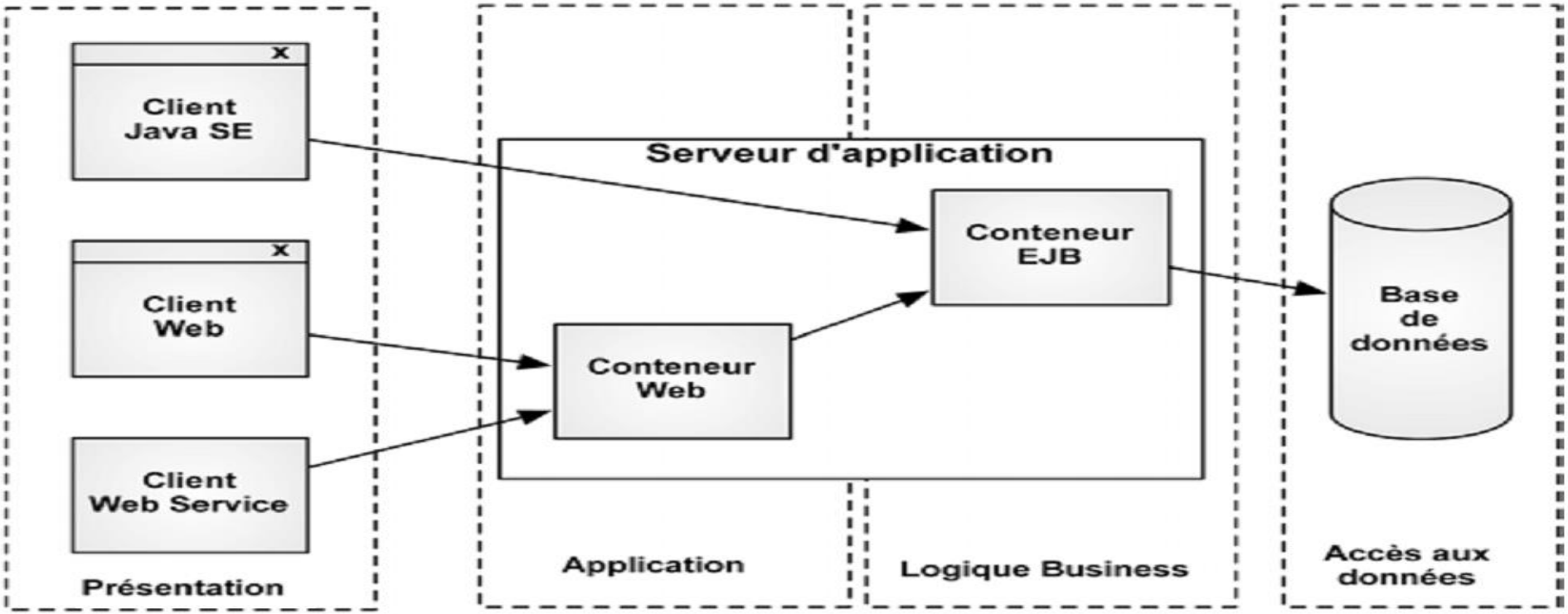


DÉTAILS DES COUCHES

Couche présentation

La **couche présentation** est **liée** au **type de clients utilisés:**
Client riche: **utiliser l'ensemble des outils et des librairies** mis à disposition pour ce **type de client** (plus particulièrement **J2SE**).

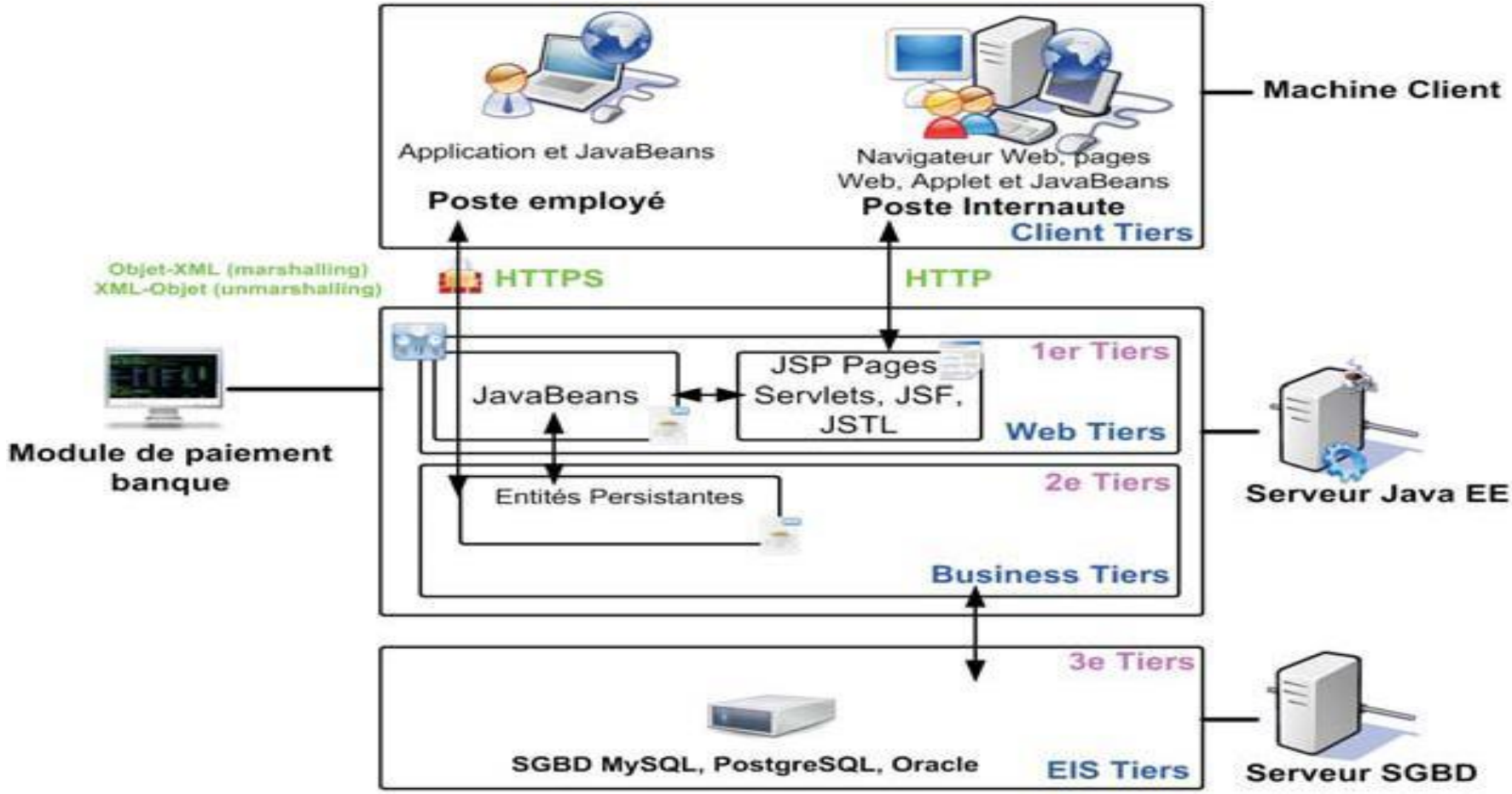
Applications web : Dans une **architecture Java EE**, on **utilise** principalement des **applications web** pour l'**interface utilisateur**.



DÉTAILS DES COUCHES

Couche présentation

Les clients riches de type application **communiquent directement avec le tiers Métier** alors que les **clients de type navigateur Internet communiquent à travers le tiers Web par l'intermédiaire de pages exécutées** sur le serveur et générées par des Servlets, JSP ou JSF.



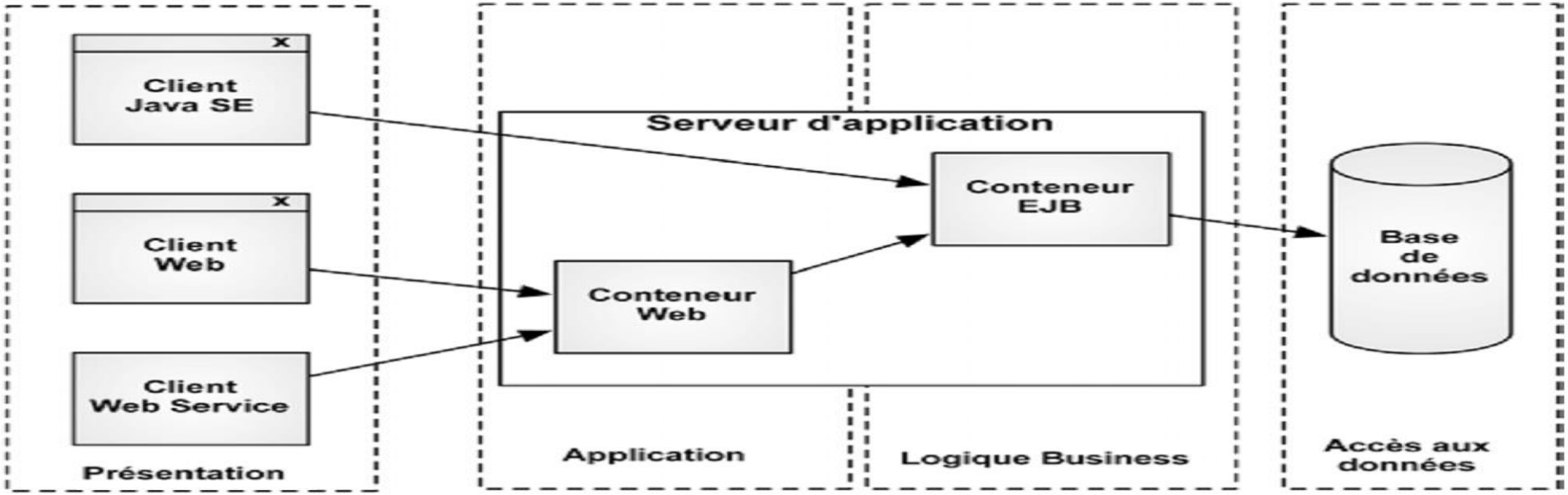
DÉTAILS DES COUCHES

Couche application

- **Médiateur** entre la **couche présentation et la couche métier**, et **contrôle l'enchaînement** des **tâches**.

- **Chargée** de **connaître** et de **gérer l'état** (connecté, en attente, déconnecté…) **des sessions des clients connectés**

- **Représente le contrôleur** dans un **modèle MVC**: Le **composant de base** utilisé dans un **environnement Java EE** est **''la servlet''.** Ce **composant est le point d'entrée à l'application**.



DÉTAILS DES COUCHES
Couche application

<u>Exemple</u>: Pour **traiter un formulaire**, vous **utiliserez** une **servlet plutôt qu'une page JSP**.

Les **pages JSP ne doivent comporter qu'un minimum de code Java** afin d'être **optimale en terme** de **maintenance**.

Cependant les **servlet/JSP** étant des **composants de bas niveau**, elles n'offrent que des **fonctionnalités de base** et **ne permettent pas** une **interconnexion simple avec les autres couches**.

Les servlet/JSP peuvent cependant **être utilisées au sein** de **framework** fournissant ces **fonctionnalités d'interconnexions**.

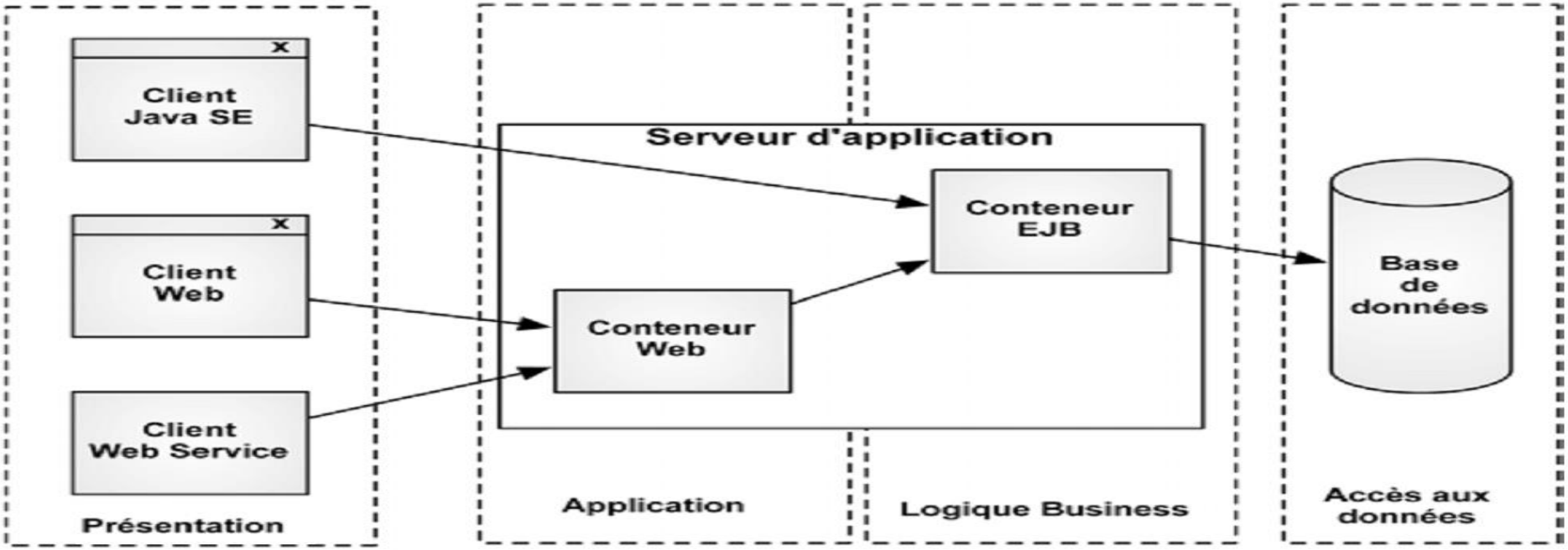
Les plus connu est ''**Struts" et Spring MVC** qui **mettent en avant** l'ensemble de l'**architecture MVC**.

De même, **les JSF** (**JavaServer Faces**) comme un **standard intégrées** dans la **plate-forme Java EE**.

DÉTAILS DES COUCHES

Couche métier

S'occuper aux **accès des différentes données qu'à leurs traitements**, suivant les processus définis par l'entreprise.



Traitement métier regroupe :

- **la vérification** de la **cohésion entre les données**,
- **l'implémentation** de la **logique métier de l'entreprise** au **niveau de l'application**,
- **la gestion du workflow**

<u>Exemple</u>: Dans le **domaine de l'assurance**, un **traitement métier** pourrait s'apparenter à une **méthode de calcul** d'une **prime d'assurance.**

Couche métier

Les composants métier ou **tiers Métier** représentent la **couche business**, **avec les données du système**, et sont de deux types :

1- **Les entités beans** (entity bean ou bean entity) : **exécutées par un conteneur léger** (pas forcément un serveur Java EE) et **permettent** de réaliser la **persistance des données** à partir des **JavaBeans** et de **Java Persistence API (JPA).**

2- **Les Enterprise JavaBeans** offrent des **possibilités plus riches** comme la **gestion des transactions**, les **accès directs par clients riches** ou encore la **gestion automatique des sessions utilisateur**, mais sont **exécutés** sur un **conteneur lourd**, c'est-à-dire **compatible Java EE.**

Les conteneurs Java EE

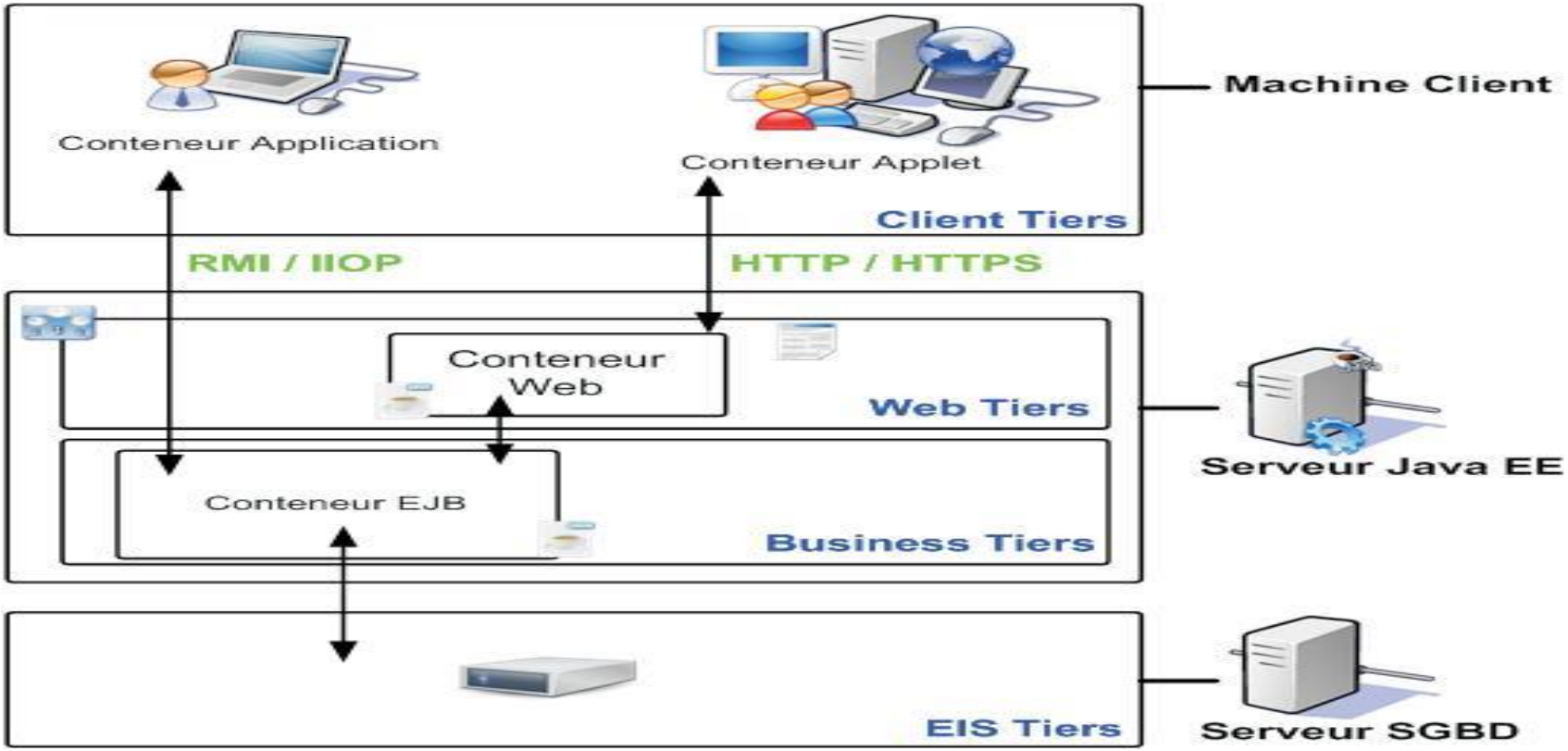
Les serveurs Java EE proposent **plusieurs types de conteneurs** (containers) pour chaque type de composant.

Chaque conteneur offre un **ensemble de services** pour les développeurs:
- **L'annuaire de nommage** d'**accès aux ressources** : **Java Naming and Directory Interface (JNDI)** est une **interface unifiée** de **gestion de nommage** pour **les services et l'accès** à ceux-ci par des applications.
- **L'injection dynamique de ressources**.
- **La gestion des accès aux bases de données**.
- **Le modèle de gestion de la sécurité.**
- **Le paramétrage des transactions.**

Les conteneurs Java EE

Une application Java EE de **type Web nécessite** un **conteneur Web pour son exécution**
Alors qu'une **application utilisant les EJB nécessite** un **conteneur EJB pour son exécution**.

Chaque conteneur propose un ensemble de services avec ses avantages et ses contraintes.



API et services Java EE

1. Java Servlet (Technologie Java EE Conteneur Web)

Ce service Java EE s'**exécute dans le conteneur Web** et propose un **ensemble de classes de gestion des échanges HTTP** de **type requête réponse.**

classe `HttpServlet`

gère les requêtes **GET** ou **POST**

méthodes

- **void doGet(request, response)** : appelé lors du traitement d'une requête GET
- **void doPost(request, response)** : appelé lors du traitement d'une requête POST
- **void init()** : initialisation au démarrage (une seule fois)
- **void destroy()** : arrêt du servlet

API et services Java EE

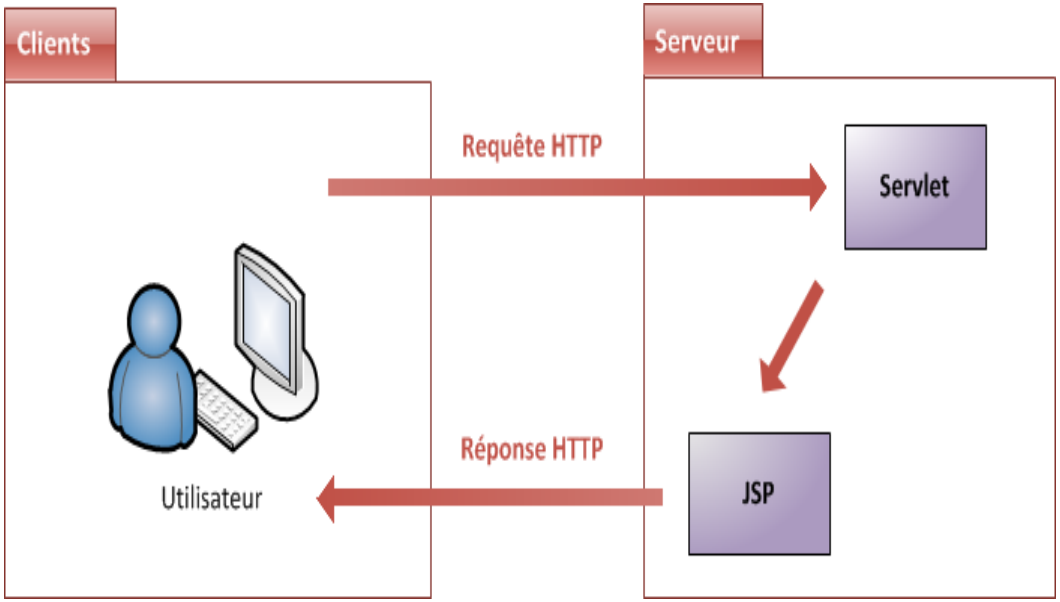
1. Java Servlet (Technologie Java EE Conteneur Web)

Ce service Java EE s'**exécute dans le conteneur Web** et propose un **ensemble de classes de gestion des échanges HTTP** de **type requête réponse.**

Application de communication Client-Serveur

<u>Solution 1:</u>
Le client peut **communiquer** des **informations à une servlet**, grâce à la **notion de paramètre**, ce qui nous amènera à **parler des formulaires HTML**.

<u>Solution 2:</u>
Le client peut **communiquer** au serveur avec des **pages JSP** (**JavaServer Pages**):
Pages HTML dans lesquelles on peut **introduire** des **séquences d'instructions Java**.



API et services Java EE

2. JavaServer Pages (JSP) (Technologie Java EE Conteneur Web)

La technologie ***JavaServer Pages (JSP)*** **s'exécute** dans le **conteneur Web** et utilise le **mécanisme de génération de Servlets** à partir de documents textuels.

Solution 2: Pages JSP

un **JSP** est une **page HTML** dans laquelle sont **introduits** des **morceaux de code Java** nommés ''**éléments de script"**

Exemple: une page JSP: fichier HTML contenant un **scriptlet (code java)**

Exemple: PageBonjour.jsp

```
<html>
<head>
<title> JSP Bonjour </title>
</head>
<body>
<% out.println ("BONJOUR") ;
%>
</body>
</html>
```

Les JSP sont **enregistrés** avec **l'extension .jsp**.
Pour pouvoir **exéctuer un JSP**, **une machine serveur** doit **disposer** d'un **interpréteur de JSP** (ou "**moteur de JSP**").

Une **requête pour l'appel de la page jsp:**

http://localisationServeur/localisationJSP/nomJSP.jsp?ParamètresEventuels

API et services Java EE

2. JavaServer Pages (JSP) (Technologie Java EE Conteneur Web)

Pages JSP

Transmission de paramètres à un JSP

Voici un **JSP** nommé **bonjourVous** donnant le **même résultat** que **notre servlet BonjourVous**

JSP bonjourVous.jsp

```
<html> <head>  <title> JSP Bonjour </title> </head>
<body>
<%
String nom = request.getParameter ("prenom") ;
if (nom == null)
{
out.println ("BONJOUR") ;
}
else
{
out.println ("BONJOUR " + nom)
}
%>
</body> </html>
```

API et services Java EE

3. JavaServer Pages Standard Tag Library (JSTL) (Technologie Java EE Conteneur Web)

La **technologie JavaServer Pages Tag Library (JSTL)** est encapsulée dans des pages JSP et propose une **librairie de balises XHTML standardisées**. **Ces balises** proposent de **gérer les conditions et itérations**, **l'internationalisation**, **les affichages HTML ou encore l'accès aux bases de données.**

4. JavaServer Faces (JSF) (Technologie Java EE Conteneur Web)

La technologie JavaServer Faces s'exécute dans **le conteneur Web. JSF 2.0** est un **framework de développement** de **projets Web**

API et services Java EE

5. Java DataBase Connectivity (JDBC) (Technologie Java SE - Conteneur Web, Conteneur EJB, Conteneur Application)

Pour gérer les commandes SQL et les dialogues avec les différents SGBD relationnels.

JDBC est utilisée lorsque nous devons accéder à une base de données dans une Servlet, page JSP, page JSF ou autres.

L'API JDBC propose un ensemble de classes et interfaces pour se connecter aux différents SGBD du marché mais également les paquetages de manipulation des données.

JDBC est une technologie sous-jacente à JPA.

API et services Java EE

6. Java Persistence API (JPA) (Technologie Java SE Conteneur Web, Conteneur EJB et Conteneur Application)

Java Persistence API (JPA) est un **standard Java** utilisé pour la **persistance des données**.

Mécanisme JPA : Utilise le **mapping objet/relationnel** et **relationnel/objet** pour **stocker les objets dans les BDs** et **inversement** de pouvoir **lire les données relationnelles** et les **transformer en objets**.

L'API Java Persistence propose **les services** suivants :

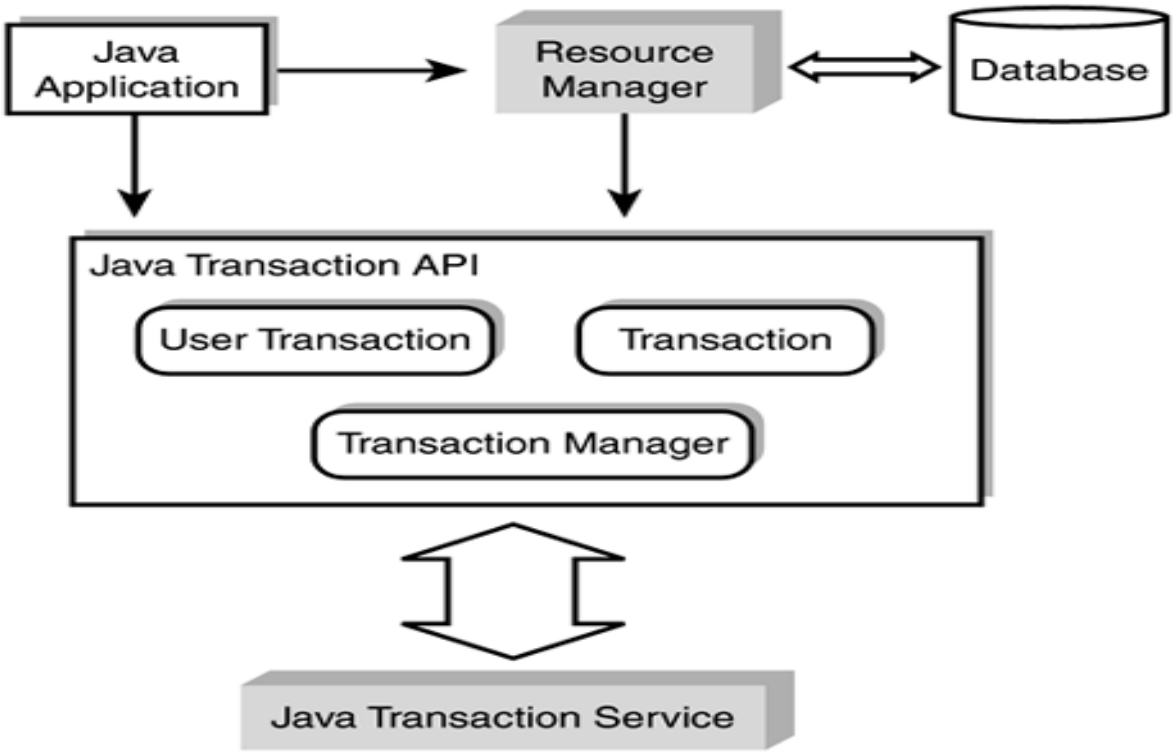
- **La gestion de la persistance**.
- **Un langage de requêtage** évolué : **Java Persistence Query Language (JPQL).**

API et services Java EE

7. Java Transaction API (JTA) (Technologie Java EE - Conteneur Web, Conteneur)

Java Transaction API (JTA) fournit un standard de **gestion des transactions** pour une **architecture Java EE**

<u>Mécanisme de JTA</u> : propose par défaut le **mode autocommit** pour la **gestion des transactions**



API et services Java EE

8. Enterprise JavaBeans (EJB) (Technologie Java EE - Conteneur EJB)

Les **composants Enterprise JavaBeans** s'exécutent dans le **conteneur (couche ou tiers Métier)**: sont des **classes** composées de **champs et de méthodes d'accès**.

Java EE propose **trois familles d'EJB:**

Les Session beans représentent les **conversations entre le client et le serveur**, qu'elles soient **avec ou sans état**.

Les beans entité (les entity beans): assurent la persistance des données

Les Message Driven Bean (MDB) **mélange** un **session bean et un écouteur de message** permettant ainsi la réception asynchrone de messages.

9- Autres Services: Java Message Service API (JMS) , JavaMail, …

Les outils préconisés par Oracle

Oracle préconise un **ensemble d'outils** pour les applications.

Un des **principaux avantages de Java** concerne **sa portabilité** et c'est ainsi que **n'importe quel serveur Java EE compatible** est **utilisable**, comme par exemple : **GlassFish, Websphere, JBoss ou autres**.

L'implémentation de la **persistance JPA** supportant les outils **EclipseLink, Hibernate, ou encore OpenJPA**.

Les outils préconisés par Oracle

Le consortium Java EE Oracle propose un **ensemble d'outils** recommandés en fonction de chaque service :

● **Le kit de développement** proposé par Oracle/Sun Microsystems **JDK 1.6** et supérieur.

● **Le serveur Java EE GlassFish v3 ou plus** compatible Java EE 6.

● **L'implémentation de persistance JPA**, **EclipseLink**.

● **Le SGBD** intégré en **standard avec le serveur Java EE** nommé **JavaDB (SGBD Derby).**

● **L'outil de développement** également appelé **IDE, NetBeans**.

Remarque:

Ces outils recommandés par Oracle seront ainsi utilisés car ils sont **compatibles, stables et conformes** à toutes les **spécifications Java EE 6.**